

Características Avanzadas de PostgreSQL para Sistemas Híbridos OLTP/OLAP: Tipos de Datos Especializados, Extensiones y Técnicas de Modelado Aplicadas al Dataset Olist

Equipo Desarrollo

Fredy Orlando Pulido Quintero

Myriam Andrea Martinez Fontecha

Nicolas Felipe Torres Amaya

Juan Francisco Javier Perez Rivero

Investigación Formativa

Maestría en Arquitectura de Software

Universidad de La Sabana

Diseño y Optimización de Bases de Datos

Profesor: Miguel Alfonso Varela

Bogotá D.C., Colombia · Mayo de 2026

Resumen

Este documento presenta una investigación formativa sobre las características avanzadas de PostgreSQL 16 que extienden su capacidad más allá de un sistema relacional convencional, con aplicación práctica al proyecto Ecommify basado en el dataset Brazilian E-Commerce de Olist. La investigación se estructura en tres ejes complementarios: (a) tipos de datos nativos especializados —JSONB, ARRAY, hstore, composite types y rangos temporales— con criterios para decidir entre estructuras desnormalizadas en JSONB y columnas relacionales tradicionales; (b) extensiones del ecosistema PostgreSQL, particularmente PostGIS para cálculos geoespaciales aplicados a la optimización de costos de envío, pg_trgm para búsqueda de productos tolerante a errores tipográficos, y las extensiones de soporte hstore y pgcrypto; y (c) técnicas de modelado híbrido OLTP/OLAP que combinan particionamiento por RANGO temporal, vistas materializadas para dashboards analíticos, triggers de mantenimiento de campos updated_at, y estrategias de mantenimiento programado (VACUUM, REFRESH, creación de particiones). Para cada eje se documentan ventajas, casos de uso, scripts DDL preparatorios y decisiones de adopción justificadas en función de los hallazgos del análisis exploratorio de datos (EDA) previo del dataset Olist, manteniendo la coherencia con la arquitectura híbrida PostgreSQL+MongoDB definida para el proyecto.

Palabras clave: PostgreSQL, JSONB, PostGIS, pg_trgm, OLTP, OLAP, particionamiento, vistas materializadas, Olist, Ecommify.

Introducción

PostgreSQL ha evolucionado desde su origen como sistema relacional clásico hacia una plataforma de datos extensible que incorpora capacidades habitualmente asociadas con sistemas NoSQL (almacenamiento de documentos JSON), bases de datos geoespaciales (PostGIS) y motores analíticos (vistas materializadas, particionamiento declarativo). El PostgreSQL Global Development Group (2025) documenta más de cuarenta tipos de datos nativos y un sistema de extensiones que permite agregar funcionalidades sin modificar el núcleo del motor. Esta riqueza convierte a PostgreSQL en una opción atractiva para arquitecturas híbridas donde un mismo motor debe atender simultáneamente cargas transaccionales (OLTP) y consultas analíticas (OLAP).

El proyecto Ecommify, basado en el dataset Brazilian E-Commerce de Olist (Olist, 2018), constituye un caso de aplicación ideal para evaluar estas capacidades. El análisis exploratorio de datos (EDA) previo (Equipo, 2026) reveló características que justifican el uso de tipos avanzados: variabilidad estructural en el catálogo de productos por categoría (apto para JSONB), distribución temporal concentrada en 2017-2018 (apto para particionamiento RANGE), demanda de consultas analíticas frecuentes sobre la tabla orders (apto para vistas materializadas) y la necesidad de cálculos geoespaciales para estimación de costos de envío (apto para PostGIS).

Este documento estructura sus resultados en tres secciones principales que responden a los tres ejes de la investigación formativa: tipos de datos avanzados, extensiones de PostgreSQL, y técnicas de modelado híbrido OLTP/OLAP. Cada sección incluye fundamentos teóricos, aplicación práctica al proyecto Ecommify, tablas comparativas y scripts DDL preparatorios.

Sección 1: Tipos de Datos Avanzados en PostgreSQL 16

1.1 Marco Conceptual

Los tipos de datos avanzados en PostgreSQL extienden el modelo relacional clásico para representar estructuras semiestructuradas (JSONB), colecciones ordenadas (ARRAY), pares clave-valor (hstore), estructuras compuestas tipo registro (composite types) y rangos continuos (range types). Cada uno responde a un patrón específico que, de implementarse mediante normalización estricta, requeriría tablas adicionales con su correspondiente costo de JOIN.

Tabla 1

Tipos de Datos Avanzados de PostgreSQL 16

Tipo	Sintaxis	Estructura	Caso de uso típico	Índice soportado
JSONB	JSONB	Documento JSON binario; permite consultas por path, operadores @>, ?, ? , ?&	Atributos variables por categoría de producto; logs de auditoría; configuraciones	GIN sobre @>, ? (jsonb_path_ops)
ARRAY	TEXT[], INTEGER[]	Colección ordenada de elementos del mismo tipo; soporta multidimensión	URLs de fotos de productos; tags; números secuenciales asociados	GIN para operadores @>, &&,
hstore	hstore	Pares clave-valor (string→string) en un solo campo	Metadatos planos sin estructura jerárquica (legacy; JSONB lo reemplaza)	GiST y GIN sobre hstore
composite	CREATE TYPE address AS (...)	Registro multicampo con tipos heterogéneos; reutilizable en columnas	Dirección compuesta (calle, ciudad, código), coordenadas (lat,lng)	B-tree sobre campos individuales
tsrange	TSTZRANGE	Rango continuo con límites abiertos/cerrados sobre TIMESTAMPTZ	Períodos de promoción, ventanas de descuento, vigencia de tarifas	GiST sobre operadores @>, &&
numrange	NUMRANGE	Rango numérico continuo con límites abiertos/cerrados	Rangos de precios; rangos de descuento por volumen; rangos etarios	GiST sobre operadores @>, &&

Nota. Elaboración propia basada en PostgreSQL Global Development Group (2025). GIN = Generalized Inverted Index. GiST = Generalized Search Tree.

1.2 JSONB vs. Columnas Normalizadas: Criterios de Decisión

La decisión entre almacenar un atributo como columna normalizada o como campo dentro de un documento JSONB no es arbitraria. JSONB ofrece flexibilidad de esquema y soporte de consultas sobre estructuras anidadas, pero penaliza el rendimiento de operaciones que requieran filtros sobre múltiples campos simultáneamente o JOINS. La Tabla 2 resume los criterios para esta decisión.

Tabla 2

JSONB vs. Columnas Normalizadas — Criterios de Decisión

Dimensión	→ Columnas normalizadas	→ JSONB
Variabilidad de atributos	Mismos campos en todas las filas (esquema fijo).	Atributos varían por fila o por subtipo (esquema flexible).
Cardinalidad de consulta	Filtros frecuentes sobre múltiples campos con índices B-tree.	Consultas ocasionales sobre campos específicos del documento.
Restricciones de integridad	CHECK, NOT NULL, FK aplicables por columna.	Validación se delega a la aplicación o a CHECK con jsonb_typeof.
JOINS y agregaciones	JOINS eficientes con índices sobre claves foráneas.	JOINS costosos (requieren extracción del campo); agregaciones limitadas.
Evolución del esquema	ALTER TABLE costoso en tablas grandes.	Agregar/quitar atributos no requiere ALTER TABLE.
Almacenamiento	Compacto; sin duplicación de claves.	Cada documento incluye nombres de claves (mayor tamaño).
Caso típico Ecommify	order_status, price, customer_state, payment_type.	product_specifications (atributos variables por categoría).

Nota. Elaboración propia basada en PostgreSQL Global Development Group (2025) y análisis del dataset Olist (Equipo, 2026).

1.3 Aplicación Práctica al Dataset Olist

La Tabla 3 documenta, para cada campo candidato de Ecommify, qué tipo avanzado se justifica adoptar y la razón técnica. Los campos transaccionales críticos (order_id, payment_value, etc.) permanecen como columnas tipadas tradicionales por razones de integridad y rendimiento de JOIN.

Tabla 3

Aplicación de Tipos Avanzados PostgreSQL en Ecommify

Campo / Atributo	Tabla	Tipo avanzado	Justificación
product_specifications	products	JSONB	Cada categoría tiene atributos distintos (TV: pulgadas, HDR; zapatos: talla, material). Esquema fijo sería ineficiente con muchas columnas NULL.
product_photos	products	TEXT[]	Lista ordenada de URLs (1-N fotos por producto). Array preserva orden y permite operadores @> sin tabla extra.
product_tags	products	TEXT[]	Etiquetas de búsqueda no jerárquicas. Operador && permite "OR" de tags con índice GIN.
promotion_period	promotions (nueva)	TSTZRANGE	Vigencia de descuento. Operadores @>, && permiten consultas como "promociones activas hoy" con índice GiST.
price_range_discount	promotion_rules	NUMRANGE	Descuentos por rango de precio (ej: 100-500 BRL = 10% off). Sintaxis nativa para inclusión.
shipping_address	orders	COMPOSITE	Reutilización: address(street, city, state, zip). Mismo tipo en customers y orders.
audit_log_changes	audit_logs (nueva)	JSONB	Cambios antes/después por columna modificada. Schema dinámico según tabla afectada.
order_status_history	orders	JSONB[]	Array de transiciones de estado con timestamp. Combina ARRAY + JSONB para historial sin tabla extra.

Nota. Algunos campos (`promotion_period`, `audit_log_changes`) requieren la creación de tablas nuevas no presentes en el dataset Olist original; corresponden a extensiones funcionales de Ecommify. Elaboración propia (Equipo, 2026).

1.4 DDL Preparatorio y Consultas que Aprovechan los Tipos Avanzados

Los siguientes ejemplos ilustran la sintaxis de declaración y consulta para cada tipo avanzado en el contexto de Ecommify.

1.4.1 JSONB en products

```
-- Extensión de products con especificaciones flexibles
ALTER TABLE products
  ADD COLUMN specifications JSONB,
  ADD COLUMN photos TEXT[],
  ADD COLUMN tags TEXT[];

-- Índice GIN para búsquedas eficientes en JSONB
CREATE INDEX idx_products_specs_gin
  ON products USING GIN (specifications jsonb_path_ops);

-- Índice GIN para arrays de tags
CREATE INDEX idx_products_tags_gin
  ON products USING GIN (tags);

-- Inserción con JSONB y array
UPDATE products SET
  specifications = '{"screen_inches": 55, "resolution": "4K", "hdr":
true}':::jsonb,
  photos = ARRAY['url1.jpg', 'url2.jpg', 'url3.jpg'],
  tags = ARRAY['smart-tv', '4k', 'samsung']
WHERE product_category_name = 'electronics';

-- Consulta: TVs 4K con HDR
SELECT product_id, specifications->>'screen_inches' AS pulgadas
FROM products
WHERE specifications @> '{"resolution": "4K", "hdr": true}':::jsonb;

-- Consulta: productos con tag específico
SELECT product_id, tags
```



```
FROM products
WHERE tags && ARRAY['samsung', 'lg']; -- intersección no vacía
```

1.4.2 TSTZRANGE para Promociones

```
-- Tabla de promociones con rango temporal
CREATE TABLE promotions (
    promotion_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    product_id      UUID REFERENCES products(product_id),
    discount_pct    NUMERIC(5,2) CHECK (discount_pct BETWEEN 0 AND 100),
    period          TSTZRANGE NOT NULL,
    EXCLUDE USING GIST (product_id WITH =, period WITH &&) -- no overlaps
);

-- Índice GiST para consultas temporales
CREATE INDEX idx_promo_period_gist ON promotions USING GIST (period);

-- Inserción: descuento del 15% para Black Friday 2018
INSERT INTO promotions (product_id, discount_pct, period)
VALUES ('a-uuid-here', 15.00,
        tstzrange('2018-11-23 00:00 -03', '2018-11-26 23:59 -03', '[]'));

-- Consulta: ¿qué promociones están activas ahora?
SELECT promotion_id, product_id, discount_pct
FROM promotions
WHERE period @> CURRENT_TIMESTAMP;
```

1.4.3 Composite Type para Dirección

```
-- Definición del tipo compuesto
CREATE TYPE address AS (
    street    TEXT,
    city      VARCHAR(60),
    state     CHAR(2),
    zip_code  INTEGER,
    country   CHAR(2)
);

-- Uso en tablas
ALTER TABLE customers
    ADD COLUMN shipping_address address;

-- Inserción con sintaxis ROW
UPDATE customers
SET shipping_address = ROW('Av. Paulista 1500', 'São Paulo', 'SP', 1310, 'BR')
WHERE customer_id = 'a-uuid-here';

-- Consulta accediendo a campo individual
SELECT customer_id, (shipping_address).city, (shipping_address).state
FROM customers WHERE (shipping_address).state = 'SP';
```


Sección 2: Extensiones de PostgreSQL Aplicables a Ecommify

2.1 Marco Conceptual de Extensiones

Las extensiones son paquetes de funcionalidad cargables que amplían PostgreSQL sin requerir modificación del código fuente del motor. Se instalan con la instrucción CREATE EXTENSION nombre y aportan nuevos tipos de datos, funciones, operadores e índices (PostgreSQL Global Development Group, 2025). La Tabla 4 resume las cuatro extensiones evaluadas para Ecommify.

Tabla 4

Extensiones de PostgreSQL Evaluadas para Ecommify

Extensión	Propósito	Funciones / Operadores clave	Caso de uso Ecommify	Decisión
PostGIS	Tipos y funciones geoespaciales (geography, geometry)	ST_Distance, ST_DWithin, ST_MakePoint, GiST sobre coords	Cálculo de distancia cliente↔depósito para estimar costo de envío	ADOPTAR
pg_trgm	Búsqueda de texto por trigramas (fuzzy match)	similarity(), %, <-> operador; índices GIN/GiST	Búsqueda de productos tolerante a errores tipográficos en barra de búsqueda	ADOPTAR
pgcrypto	Funciones criptográficas (hash, encrypt, decrypt)	crypt(), gen_salt(), digest(), pgp_sym_encrypt()	Hashing de contraseñas de usuarios (si Ecommify maneja autenticación)	ADOPTAR
hstore	Almacenamiento clave-valor plano	hstore_to_json(), operadores ->, ?, %	Sustituible por JSONB para nuevos desarrollos	NO ADOPTAR
pgcrypto	UUID generation	gen_random_uuid() (provisto por pgcrypto en PG ≤ 12)	PostgreSQL 16 incluye gen_random_uuid() nativo; pgcrypto ya cubre cifrado	YA INCLUIDA

btree_gin	Permite índices GIN sobre tipos B-tree	GIN multi-columna con tipos mixtos	Indexación combinada de JSONB + columnas escalares	EVALUAR
-----------	--	------------------------------------	--	---------

Nota. Las extensiones marcadas como "ADOPTAR" se incluyen en el script de inicialización de la base de datos. Elaboración propia (Equipo, 2026).

2.2 PostGIS — Cálculo de Distancia para Costos de Envío

PostGIS aporta los tipos GEOGRAPHY (cálculos sobre esfera, en metros) y GEOMETRY (cálculos planares, más rápidos pero imprecisos en grandes distancias). Para Ecommify, el cálculo de distancia entre el depósito y la dirección del cliente es directo con el tipo GEOGRAPHY. El módulo de envío puede usar la distancia calculada como input para una función de costo. Nota: en este proyecto Ecommify no se asigna geolocalización a los vendedores (sellers) inicialmente; solo se geolocaliza al cliente y eventualmente al depósito central.

```
-- Instalación de PostGIS
CREATE EXTENSION IF NOT EXISTS postgis;

-- Tabla auxiliar de depósitos (warehouse) con punto geográfico
CREATE TABLE warehouses (
    warehouse_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name            VARCHAR(120) NOT NULL,
    location        GEOGRAPHY(POINT, 4326) NOT NULL -- WGS84 (lat/lng)
);

-- Índice GiST para búsquedas espaciales
CREATE INDEX idx_warehouses_loc_gist ON warehouses USING GIST (location);

-- Inserción: depósito central en São Paulo
INSERT INTO warehouses (name, location)
VALUES ('Depósito SP Centro', ST_MakePoint(-46.6333, -23.5505)::geography);

-- Cálculo de distancia desde cliente a depósito más cercano
SELECT c.customer_id,
       w.warehouse_id,
       ST_Distance(
           ST_MakePoint(g.geolocation_lng, g.geolocation_lat)::geography,
           w.location
```

```

        ) / 1000.0 AS distance_km
FROM   customers c
JOIN   geolocation g ON g.zip_code_prefix = c.zip_code_prefix
CROSS JOIN LATERAL (
    SELECT warehouse_id, location
    FROM   warehouses
    ORDER BY location <-> ST_MakePoint(g.geolocation_lng,
g.geolocation_lat)::geography
    LIMIT 1
) AS w;

```

2.3 pg_trgm — Búsqueda Tolerante a Errores Tipográficos

pg_trgm descompone cadenas en trigramas (subsecuencias de 3 caracteres) y permite calcular la similitud entre dos cadenas mediante la función `similarity()`. Es ideal para barras de búsqueda donde el usuario puede cometer errores tipográficos. Para Ecommify, se aplica a la búsqueda de productos por nombre o categoría con tolerancia a errores como "telefono" vs "teléfono" o "samsung" vs "samsung".

```

-- Instalación de pg_trgm
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Índice GIN sobre nombre de categoría (tabla products)
CREATE INDEX idx_products_category_trgm
    ON products USING GIN (product_category_name gin_trgm_ops);

-- Búsqueda tolerante a errores: "telephoni" buscará "telephony"
SELECT product_id,
       product_category_name,
       similarity(product_category_name, 'telephoni') AS sim
FROM   products
WHERE  product_category_name % 'telephoni'    -- umbral por defecto 0.3
ORDER BY sim DESC
LIMIT 10;

-- Configurar umbral de similitud
SET pg_trgm.similarity_threshold = 0.4;

-- Operador <-> retorna distancia (1 - similarity)
SELECT product_category_name
FROM   products
ORDER BY product_category_name <-> 'casa_decoracon'    -- typo de 'casa_decoracao'
LIMIT 5;

```

2.4 pgcrypto — Cifrado y Hashing

pgcrypto provee funciones criptográficas estándar incluyendo hashing seguro de contraseñas con bcrypt, cifrado simétrico con PGP y generación de hashes de mensaje. En Ecommify, su uso principal es el hashing de contraseñas de cuentas de usuario y el cifrado de campos sensibles (números de documento, datos de tarjeta tokenizados) en tablas que la aplicación requiera.

```
-- Instalación de pgcrypto
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Hashing de contraseña con bcrypt (cost 12)
CREATE TABLE app_users (
    user_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email          CITEXT UNIQUE NOT NULL,
    password_hash  TEXT NOT NULL,
    created_at     TIMESTAMPTZ DEFAULT NOW()
);

-- Inserción con password hash bcrypt
INSERT INTO app_users (email, password_hash)
VALUES ('user@ecommify.co',
       crypt('mypassword_xyz', gen_salt('bf', 12)));

-- Verificación de contraseña al login
SELECT user_id
FROM   app_users
WHERE  email = 'user@ecommify.co'
      AND password_hash = crypt('mypassword_xyz', password_hash);

-- Cifrado simétrico de campo sensible
UPDATE customers
SET    document_number_enc = pgp_sym_encrypt(document_number, 'secret_key')
WHERE  document_number IS NOT NULL;
```

Sección 3: Técnicas de Modelado Híbrido OLTP/OLAP

3.1 Requisitos OLTP vs. OLAP de Ecommify

Un sistema OLTP (Online Transaction Processing) prioriza la inserción rápida y atómica de transacciones pequeñas con alta concurrencia (compra de un pedido, registro de un pago). Un sistema OLAP (Online Analytical Processing) prioriza consultas agregadas sobre grandes volúmenes históricos (ventas por mes, ranking de productos). En Ecommify, ambos patrones coexisten sobre la misma base de datos PostgreSQL: el frontend transaccional genera escrituras OLTP, y los dashboards ejecutivos consumen consultas OLAP. La Tabla 5 caracteriza ambos perfiles de carga.

Tabla 5

Comparación de Requisitos OLTP vs. OLAP en Ecommify

Dimensión	OLTP (Transaccional)	OLAP (Analítico)
Operaciones predominantes	INSERT, UPDATE puntuales por order_id	SELECT con agregaciones, GROUP BY, JOINs múltiples
Volumen por operación	Filas individuales o decenas	Miles o millones de filas escaneadas
Latencia esperada	< 100 ms p95 por operación	< 5 s por dashboard
Concurrencia	Cientos de transacciones simultáneas	Decenas de consultas simultáneas (usuarios analistas)
Patrón temporal	Acceso reciente (últimos días)	Histórico completo (años)
Optimización ideal	Índices B-tree estrechos en columnas clave	Vistas materializadas, particionamiento, columnar (TimescaleDB)
Caso típico Ecommify	INSERT INTO orders al confirmar compra	mv_sales_by_category_monthly para dashboard ejecutivo

Nota. Elaboración propia basada en perfiles de carga observados en el dataset Olist (Equipo, 2026).

3.2 Particionamiento RANGE de la Tabla orders

El particionamiento declarativo divide una tabla lógica en sub-tablas físicas según el valor de una columna de partición. En PostgreSQL 16, el planificador realiza partition pruning automático, limitando las consultas a las particiones relevantes. Para Ecommify, el EDA reveló que el 95% de los pedidos se concentran en 2017-2018, justificando particiones anuales con una estrategia hot/cold: las particiones recientes (hot) permanecen en almacenamiento rápido, mientras las antiguas (cold) pueden moverse a almacenamiento más económico cuando el dataset crezca.

```
-- Tabla particionada por RANGE
CREATE TABLE orders (
    order_id                UUID NOT NULL,
    customer_id             UUID NOT NULL REFERENCES customers(customer_id),
    order_status            VARCHAR(20) NOT NULL,
    order_purchase_timestamp TIMESTAMPTZ NOT NULL,
    /* otros campos */
    PRIMARY KEY (order_id, order_purchase_timestamp)
) PARTITION BY RANGE (order_purchase_timestamp);

-- Partición HOT (año actual): almacenamiento rápido (default)
CREATE TABLE orders_2018 PARTITION OF orders
    FOR VALUES FROM ('2018-01-01') TO ('2019-01-01');

-- Partición HOT (año anterior)
CREATE TABLE orders_2017 PARTITION OF orders
    FOR VALUES FROM ('2017-01-01') TO ('2018-01-01');

-- Partición COLD (archivo): puede moverse a tablespace lento
CREATE TABLE orders_2016 PARTITION OF orders
    FOR VALUES FROM ('2016-01-01') TO ('2017-01-01')
    TABLESPACE cold_storage; -- opcional: tablespace separado

-- Partición DEFAULT para fechas fuera de rango definido
CREATE TABLE orders_default PARTITION OF orders DEFAULT;

-- Índices: PostgreSQL los crea automáticamente en cada partición
CREATE INDEX idx_orders_customer ON orders (customer_id);
CREATE INDEX idx_orders_status   ON orders (order_status)
```



```
WHERE order_status != 'delivered';    -- índice parcial
```

La creación automática de nuevas particiones se gestiona con un job mensual programado mediante `pg_cron` (extensión externa) o `cron` del sistema operativo. Cada inicio de mes, el job crea la partición correspondiente al mes futuro para evitar que pedidos nuevos caigan en la partición `DEFAULT`.

3.3 Vistas Materializadas para Dashboards

Una vista materializada (MV) almacena el resultado precomputado de una consulta como una tabla física. A diferencia de las vistas regulares (que se ejecutan en cada acceso), las MVs permiten consultas sub-segundo sobre agregaciones complejas. El trade-off es que los datos están "frescos al último refresh"; para Ecommify, un refresh semanal de las MVs analíticas es adecuado para dashboards ejecutivos. Para refrescos sin bloqueo de lectura se usa `REFRESH MATERIALIZED VIEW CONCURRENTLY`, que requiere un `UNIQUE INDEX` en la MV.

```
-- MV 1: Ventas mensuales por categoría
CREATE MATERIALIZED VIEW mv_sales_by_category_monthly AS
SELECT
    date_trunc('month', o.order_purchase_timestamp) AS year_month,
    pcnt.product_category_name_english              AS category,
    COUNT(DISTINCT o.order_id)                     AS orders_count,
    SUM(oi.price + oi.freight_value)                AS gross_revenue,
    AVG(oi.price)                                   AS avg_price
FROM    orders o
JOIN    order_items oi ON oi.order_id = o.order_id
JOIN    products p      ON p.product_id = oi.product_id
JOIN    product_category_name_translation pcnt
        ON pcnt.product_category_name = p.product_category_name
WHERE   o.order_status = 'delivered'
GROUP BY 1, 2
WITH NO DATA;

-- Índice único requerido para REFRESH CONCURRENTLY
CREATE UNIQUE INDEX uidx_mv_sales_cat_month
    ON mv_sales_by_category_monthly (year_month, category);
```

```
-- Primer refresh (popula los datos)
REFRESH MATERIALIZED VIEW mv_sales_by_category_monthly;

-- Refresh posterior sin bloquear lecturas (uso semanal)
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_sales_by_category_monthly;

-- -----
-- MV 2: Segmentos RFM de clientes (Recency, Frequency, Monetary)
CREATE MATERIALIZED VIEW mv_customer_segments AS
WITH base AS (
    SELECT
        o.customer_id,
        MAX(o.order_purchase_timestamp)           AS last_order,
        COUNT(DISTINCT o.order_id)                 AS freq,
        SUM(op.payment_value)                       AS monetary
    FROM orders o
    JOIN order_payments op ON op.order_id = o.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY 1
),
rfm AS (
    SELECT *,
        NTILE(5) OVER (ORDER BY last_order DESC) AS r,
        NTILE(5) OVER (ORDER BY freq           DESC) AS f,
        NTILE(5) OVER (ORDER BY monetary       DESC) AS m
    FROM base
)
SELECT customer_id, r, f, m, (r*100 + f*10 + m) AS rfm_score,
CASE
    WHEN r=1 AND f<=2 AND m<=2 THEN 'champion'
    WHEN r<=2 AND f<=2         THEN 'loyal'
    WHEN r>=4                   THEN 'at_risk'
    ELSE 'regular'
END AS segment
FROM rfm
WITH NO DATA;

CREATE UNIQUE INDEX uidx_mv_segments ON mv_customer_segments (customer_id);
```

3.4 Triggers para Mantenimiento de updated_at

PostgreSQL no provee una pseudo-columna ON UPDATE como MySQL. El patrón estándar para mantener un campo updated_at consiste en crear una función trigger reutilizable y aplicarla a cada tabla que lo requiera. Este patrón es esencial para CDC (Change Data Capture) y para alimentar ETL incrementales hacia el pipeline MongoDB orders_summary.

```
-- Función trigger reutilizable
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS TRIGGER AS $$
```

```

BEGIN
    NEW.updated_at := NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Aplicar a la tabla orders
ALTER TABLE orders ADD COLUMN updated_at TIMESTAMPTZ DEFAULT NOW();
CREATE TRIGGER trg_orders_updated_at
    BEFORE UPDATE ON orders
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- Aplicar a products
ALTER TABLE products ADD COLUMN updated_at TIMESTAMPTZ DEFAULT NOW();
CREATE TRIGGER trg_products_updated_at
    BEFORE UPDATE ON products
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- Aplicar a customers, sellers, etc.
-- (mismo patrón)

-- Índice para consultas ETL incrementales
CREATE INDEX idx_orders_updated_at ON orders (updated_at);
    
```

3.5 Estrategia de Mantenimiento y Monitoreo

Tabla 6

Jobs Programados de Mantenimiento PostgreSQL para Ecommify

Job	Frecuencia	Comando / Script	Propósito	Métrica
VACUUM ANALYZE	Diario 03:00	VACUUM (ANALYZE) tables_critical;	Recuperar espacio de filas muertas; actualizar estadísticas del planificador	autovacuum_count
REFRESH MV semanal	Domingo 02:00	REFRESH MV CONCURRENTLY mv_sales_by_cat;	Actualizar dashboards con datos de la semana	last_refresh_time
REFRESH MV RFM	Lunes 04:00	REFRESH MV CONCURRENTLY mv_customer_segments;	Segmentación de clientes actualizada para campañas	last_refresh_time
Crear partición mensual	Día 1 cada mes	CREATE TABLE orders_YYYY_MM PARTITION OF orders ...	Evitar que nuevos pedidos caigan en partición DEFAULT	partitions_count
Mover partición a cold	Anual	ALTER TABLE orders_YYYY SET	Mover datos antiguos a almacenamiento económico	tablespace_size

		<code>TABLESPACE cold_storage;</code>		
REINDEX (mensual)	Mensual 01:00	<code>REINDEX INDEX CONCURRENTLY idx_critical;</code>	Reducir bloat en índices de alta escritura	<code>index_bloat_ratio</code>
Backup lógico	Diario	<code>pg_dump --format=custom > backup_\$(date).dump</code>	Backup transaccional consistente	<code>backup_size_mb</code>

Nota. Los jobs se ejecutan con `pg_cron` (PostgreSQL extension) o `cron` del sistema operativo.

Elaboración propia basada en PostgreSQL Global Development Group (2025).

3.6 Estrategia de Escalamiento — Límites del Plan Gratuito

El plan gratuito de Supabase ofrece 500 MB de almacenamiento, 2 vCPU compartidas y pausa automática tras una semana de inactividad. Para Ecommify, esto es suficiente durante la fase de desarrollo y evaluación académica del dataset Olist. La Tabla 7 documenta los umbrales de escalamiento que disparan una migración a infraestructura de pago.

Tabla 7

Umbrales de Escalamiento y Estrategia de Mitigación

Recurso	Límite plan gratuito	Acción de mitigación	Migración recomendada
Almacenamiento	500 MB (Supabase)	Compresión TOAST automática, mover particiones COLD a archivo externo (S3/MinIO)	Supabase Pro (\$25/mes, 8 GB)
vCPU	2 vCPU compartidas	Programar MV refresh fuera de hora pico; reducir índices innecesarios	Supabase Pro (dedicada)
Conexiones	60 conexiones simultáneas	PgBouncer con pool transaccional	Supabase Pro (200 conexiones)

Backup retention	7 días	pg_dump local + push a Google Drive automatizado	Supabase Pro (30 días)
Pausa por inactividad	1 semana sin uso	cron ping cada 6 días para mantener activa	Supabase Pro (sin pausa)

Conclusiones

Esta investigación formativa documentó tres ejes complementarios para extender PostgreSQL 16 como motor de datos híbrido para Ecommify. Las decisiones clave adoptadas son:

1. **Tipos avanzados adoptados:** JSONB para product_specifications (esquema flexible por categoría); TEXT[] para photos y tags; TSTZRANGE para promotion_period; composite type address para reutilización entre customers y orders.
2. **Extensiones adoptadas:** PostGIS para cálculo geoespacial cliente↔depósito en estimación de envío; pg_trgm para búsqueda fuzzy de productos; pgcrypto para hashing de contraseñas de aplicación.
3. **Modelado OLTP/OLAP:** particionamiento RANGE anual en orders (hot 2017-2018, cold 2016); dos vistas materializadas (mv_sales_by_category_monthly, mv_customer_segments) con refresh semanal CONCURRENT; triggers updated_at para alimentar ETL incremental a MongoDB orders_summary.
4. **Mantenimiento programado:** VACUUM ANALYZE diario, REFRESH MV semanal, creación automática de partición mensual, REINDEX mensual, backup diario con pg_dump.
5. **Escalamiento controlado:** plan Supabase gratuito durante desarrollo académico, migración a Supabase Pro al superar 500 MB de almacenamiento o 60 conexiones simultáneas.

La aplicación de estas técnicas no requiere abandonar el modelo relacional ni migrar a un motor distinto: PostgreSQL 16 ofrece las capacidades necesarias para sostener cargas OLTP y OLAP simultáneamente en un único sistema, complementado por MongoDB para las entidades de esquema flexible (reviews) y consultas geoespaciales masivas (geolocation collection) según la arquitectura híbrida previamente definida.

Referencias

Olist. (2018). *Brazilian E-Commerce public dataset by Olist* [Conjunto de datos]. Kaggle.

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

PostGIS Project Steering Committee. (2025). *PostGIS 3.4 manual*.

<https://postgis.net/documentation/>

PostgreSQL Global Development Group. (2025). *Data types. PostgreSQL 16 Documentation*.

<https://www.postgresql.org/docs/16/datatype.html>

PostgreSQL Global Development Group. (2025). *Server programming: Extending SQL*.

PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/extend.html>

PostgreSQL Global Development Group. (2025). *Table partitioning. PostgreSQL 16*

Documentation. <https://www.postgresql.org/docs/16/ddl-partitioning.html>

PostgreSQL Global Development Group. (2025). *Materialized views. PostgreSQL 16*

Documentation. <https://www.postgresql.org/docs/16/rules-materializedviews.html>

PostgreSQL Global Development Group. (2025). *pg_trgm — support for similarity of text using*

trigram matching. <https://www.postgresql.org/docs/16/pgtrgm.html>

PostgreSQL Global Development Group. (2025). *pgcrypto — cryptographic functions*.

<https://www.postgresql.org/docs/16/pgcrypto.html>

Equipo, D. (2026). *Análisis exploratorio de datos del ecosistema de e-commerce brasileño:*

Dataset Olist 2016-2018 [Informe académico]. Universidad de La Sabana.

Supabase Inc. (2025). *Platform limits*. *Supabase Documentation*.

<https://supabase.com/docs/guides/platform/limits>